UNIVERSIDADE FEDERAL DO CEARÁ

Relatório Final Modelo

Disciplina: Qualidade de Software
Equipe: Ana Letícia Oliveira Mesquita - 570889, Maria Beatriz Ribeiro de Castro - 565662
Projeto: github.com/trustedsec/hate_crack (Hate_Crack)
Fork: github.com/Beeatriz-Castro/hate_crack/tree/refatoracao-test-smells
Pull Request: https://github.com/trustedsec/hate_crack/pull/372
Commit SHA: fefe2abd9ce8de8880b471c6ad68166177ec2310

1. Apresentação do projeto

O Hate_Crack é uma ferramenta de automação de ataques de quebra de senhas (password cracking) desenvolvida pela TrustedSec. Ele funciona como um "wrapper" em torno do hashcat: em vez de o profissional de segurança (pentester / red team) montar manualmente cada comando do hashcat, o Hate_Crack oferece uma metodologia pronta com vários tipos de ataque encadeados (dicionário, máscaras, regras, PCFG, OMEN, força bruta, combinações etc.), além de integrações com serviços como Hashview e Hashmob.

O projeto é escrito em Python e possui uma suíte de testes grande e madura, concentrada na pasta tests/ (com submódulos como HashcatRosetta e pcfg_cracker trazendo seus próprios testes). Os testes usam o pytest e fazem uso intenso de mocks (unittest.mock), fixtures e marcadores (ex.: testes "live" que só rodam com variáveis de ambiente específicas). São 183 arquivos analisados e quase 3.900 testes, cobrindo desde funções utilitárias até fluxos de ponta a ponta.

2. Estado inicial do projeto

2.1 Test smells detectados antes da refatoração (TEMPY)

Preencha com base no arquivo metrics-before-tempy/resumo.json.

Test smell	Ocorrências detectadas
Conditional Test Logic	581
Unknown Test	129
Exception Handling	115
Programming Paradigms Blend	72

Test smell	Ocorrências detectadas
Verbose Test	66
Sleepy Test	52
Undefined Test	33
Non-Functional Statement	31
Redundant Print	24
Total	**1103**

2.2 Resultados dos testes e cobertura antes da refatoração (Pytest)

Métrica	Valor
Testes aprovados (Passed)	3791
Testes com falha (Failed)	6
Erros (Errors)	0
Ignorados (Skipped)	90
Cobertura total (%)	83,9%

2.3 Métricas do código de teste antes da refatoração (Radon)

Métrica	Valor
SLOC total dos testes (linha TOTAL com origem = teste)	41.099
Complexidade ciclomática média das funções de teste	2,81
Índice de manutenibilidade médio dos arquivos de teste	50,78

3. Refatorações realizadas

3.1 Seleção das ocorrências

Começamos selecionando, entre os smells detectados pelo TEMPY (e o **Assertion Roulette NÃO é detectado pelo TEMPY** — só pelo CATSPY), os tipos com correspondência no catálogo CATSPY e com estratégia de refatoração clara. Priorizamos pela severidade e pelo impacto indicados no CATSPY, cobrindo múltiplas categorias de qualidade (legibilidade, manutenibilidade, confiabilidade e eficiência):

- **Redundant Print** (confiabilidade): prints que não testam nada, poluindo a execução automatizada.
- **Conditional Test Logic** (legibilidade/manutenibilidade/confiabilidade): if/for no corpo do teste, que podem esconder caminhos não testados.

- **Verbose Test** (eficiência): métodos longos (>30 linhas do CATSPY e ≥45 linhas do TEMPY) com múltiplas responsabilidades.
- **Sleepy Test** (eficiência/confiabilidade): time.sleep fixo, que torna o teste lento e potencialmente *flaky*.
- **Exception Handling** e **Non-Functional Statement**: resolvidos de forma incidental e deliberada ao longo das refatorações.
- **Assertion Roulette** (legibilidade/manutenibilidade): múltiplas asserções sem mensagem explicativa no mesmo teste — se uma falha, não dá pra saber qual. Resolvido adicionando uma mensagem a cada assert.
- **Unknown Test** (legibilidade/manutenibilidade): testes que só verificavam exceção via pytest.raises sem asserção explícita, resolvidos adicionando uma asserção sobre a exceção.

Um ponto central da nossa seleção foi a análise crítica caso a caso (etapa "Análise e Decisão" do fluxo do CATSPY): o Hate_Crack tem testes muito bem escritos no estilo moderno do pytest, então boa parte das detecções do TEMPY são falsos positivos (ver Seção 10). Optamos por refatorar apenas ocorrências genuínas, evitando "piorar" testes já corretos — o que seria o oposto do objetivo da atividade.

Ao todo foram refatoradas 85 ocorrências de 8 tipos de test smell (acima do mínimo de 40 ocorrências / 4 tipos para dupla).

3.2 Lista das refatorações

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R01	Redundant Print (x13) + Conditional + Verbose	tests/test_hashview.py::test_create_job_workflow	a5b28d2 + b2a188f	sim	sim	não
R02	Conditional Test Logic	tests/test_hashview.py::test_upload_skips_wrong_type_line	01948c8	sim	sim	não
R03	Conditional	tests/test_hashview.py::test_upload_preser	23409d6	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
	Test Logic	ves_non_utf8_plaintext				
R04	Conditional Test Logic	tests/test_hashview.py::test_upload_preserves_hex_wrapped_valid_utf8_plaintext	bd13a4f	sim	sim	não
R05	Conditional + Verbose	tests/test_hashview.py::test_upload_retries_batch_after_server_rejects_one_plaintext	054c03b	sim	sim	não
R06	Verbose Test	tests/test_hashview.py::test_download_left_with_auto_merge	96177a0	sim	sim	não
R07	Verbose Test	tests/test_omen_attack.py::test_builds_correct_pipe_commands	4f81c87	sim	sim	não
R08	Verbose Test	tests/test_config_startup_wiring.py::test_case1_migrates_integration_keys_and_prunes_config_json	1f9b1a4 + 5c55630	sim	sim	sim (Undefined Test)
R09	Conditional + Verbose	tests/test_ntlm_preprocessing.py::test_full_pipeline_with_filtering	09d50be	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R10	Verbose Test	tests/test_version_check.py::test_run_upgrade_switches_from_dev_to_main_then_upgrades	1467058	sim	sim	não
R11	Exception Handling + Non-Functional Statement	tests/test_version_check.py::test_flag_selects_channel	1821c46	sim	sim	não
R12	Conditional Test Logic (x2)	tests/test_main_spoonman.py::test_the_corpus_is_recognized_through_a_symlink_or_a_relative_form	603615a	sim	sim	sim (Undefined Test)
R13	Verbose Test	tests/test_main_pcfg.py::test_builds_expected_subprocess	33b29f8	sim	sim	não
R14	Sleepy Test (+ Exception Handling)	tests/test_notify_tailer.py::test_seeks_to_eof_on_start	47e2d75	sim	sim	não
R15	Sleepy Test (+ Excep	tests/test_notify_tailer.py::test	a367804	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
	tion Handling)	_truncation_resets_position				
R16	Sleepy Test	tests/test_progress_spinner.py::test_spinner_yields_handle_and_renders_detail	2a9f431	sim	sim	não
R17	Conditional + Verbose	tests/test_runhcat_cmd.py::test_brain_rejection_recovers_with_the_long_form_client_flag	dcb50c3	sim	não	não
R18	Conditional + Verbose	tests/test_runhcat_cmd.py::test_brain_rejection_retries_at_most_once	0acaef7	sim	sim	não
R19	Conditional (x2) + Verbose	tests/test_smart_mask.py::test_hcatSmartMask_reports_a_slot_exhausted_template_instead_of_dropping_it_quietly	42cd65f + 2c83ad2	sim	sim	não
R20	Verbose Test	tests/test_upload_cracked_hashes.py::test_upload_cracked_hashes_preserves_plaintext_whitespace	1c5d21e	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R21	Verbose Test	tests/test_upload_cracked_hashes.py::test_upload_repairs_historical_double_encoded_ntlm_plaintext	30c7ba5 + eff5f6b	sim	sim	sim (Undefined Test)
R22	Exception Handling	tests/test_api_downloads.py::test_keyboardinterrupt_cleanup	24a7477	sim	sim	não
R23	Assertion Roulette	tests/test_api_downloads.py::test_daemon_starts_with_expected_args	725e891	sim	**N/A** (TEMPY não detecta)	
R24	Assertion Roulette	tests/test_api_downloads.py::test_list_parses_rows	725e891	sim	**N/A** (TEMPY não detecta)	
R25	Assertion Roulette	tests/test_api_downloads.py::test_list_parses_real_transmission_4_row	725e891	sim	**N/A** (TEMPY não detecta)	
R26	Assertion Roulette	tests/test_api_downloads.py::test_list_parses_two_token_have_and_eta	725e891	sim	**N/A** (TEMPY não detecta)	
R27	Assertion Roulette	tests/test_api_downloads.py::test_wait_for_all_reports_progress	725e891	sim	**N/A** (TEMPY não detecta)	

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R28	Assertion Roulette	tests/test_api_downloads.py::test_successful_download	f79dca8 + 8172598	sim	**N/A** (TEMPY não detecta)	
R29	Assertion Roulette	tests/test_api_downloads.py::test_404_on_primary_falls_back_to_official_alternate_url	e708c1d + 8172598	sim	**N/A** (TEMPY não detecta)	
R30	Assertion Roulette	tests/test_api_downloads.py::test_untrusted_url_falls_back_to_hashmob_host	5ded1db	sim	**N/A** (TEMPY não detecta)	
R31	Assertion Roulette	tests/test_api_downloads.py::test_retries_through_backoff_on_429	07eaeaf	sim	**N/A** (TEMPY não detecta)	
R32	Assertion Roulette	tests/test_api_downloads.py::test_happy_path	f995f4c	sim	**N/A** (TEMPY não detecta)	
R33	Assertion Roulette	tests/test_api_downloads.py::test_gives_up_after_max_attempts	b76825f	sim	**N/A** (TEMPY não detecta)	
R34	Assertion Roulette	tests/test_api_downloads.py::test_skips_already_downloaded_in_all_branch	419e7c5	sim	**N/A** (TEMPY não detecta)	

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R35	Assertion Roulette	tests/test_api_downloads.py::test_creates_cache_when_missing	a0b8f1a	sim	**N/A** (TEMPY não detecta)	
R36	Verbose Test	tests/test_api_downloads.py::test_two_entries_same_name_different_id_both_kept	cd57f6d	sim	sim	sim
R37	Verbose Test	tests/test_api_downloads.py::test_rate_coerced_to_float	917a184	sim	sim	não
R38	Verbose Test	tests/test_api_downloads.py::test_falsy_ids_not_deduplicated	2f424b3	sim	sim	não
R39	Conditional Test Logic	tests/test_api_downloads.py::test_three_pages_fetched_and_returned_in_order	b6d8327	sim	sim	não
R40	Conditional Test Logic	tests/test_api_downloads.py::test_same_id_on_two_pages_kept_once	95d9e11	sim	sim	não
R41	Conditional Test Logic	tests/test_api_downloads.py::test_total_pages_caps_at_last_page	58dcf34	sim	não	não
R42	Conditional	tests/test_api_downloads.py::test_real_fetch	4be7170	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
	Test Logic	_integration_all_pages_ok				
R43	Conditional Test Logic	tests/test_api_downloads.py::test_page2_of3_fails_returns_pages1and3	15152ec	sim	sim	não
R44	Conditional Test Logic	tests/test_api_downloads.py::test_page2_json_decode_error_returns_pages1and3	ad06e6f	sim	sim	não
R45	Verbose Test	tests/test_api_downloads.py::test_tier_mapping	d773e2f	sim	sim	não
R46	Unknown Test	tests/test_api_downloads.py::test_startup_timeout_raises	8e123ba	sim	sim	não
R47	Unknown Test	tests/test_api_downloads.py::test_add_raises_when_torrent_not_added	a50d401	sim	sim	não
R48	Unknown Test	tests/test_api_downloads.py::test_wait_for_all_keyboard_interrupt_propagates	21b4872	sim	sim	não
R49	Unknown Test	tests/test_api_downloads.py::test_keyboard_interrupt_propagates	b43337e	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R50	Unknown Test	tests/test_api_downloads.py::test_invalid_variant_raises_value_error	aa88071	sim	sim	não
R51	Unknown Test	tests/test_api_downloads.py::test_non_429_exception_reraises	faf2f35	sim	sim	não
R52	Conditional Test Logic	tests/test_api_downloads.py::test_real_fetch_integration_page2_fails	e607e83	sim	sim	não
R53	Conditional Test Logic	tests/test_api_downloads.py::test_multiple_candidates_tries_each	496516a	sim	sim	não
R54	Conditional Test Logic	tests/test_api_downloads.py::test_unresolvable_bare_name_not_found_message	51f9dd7	sim	sim	não

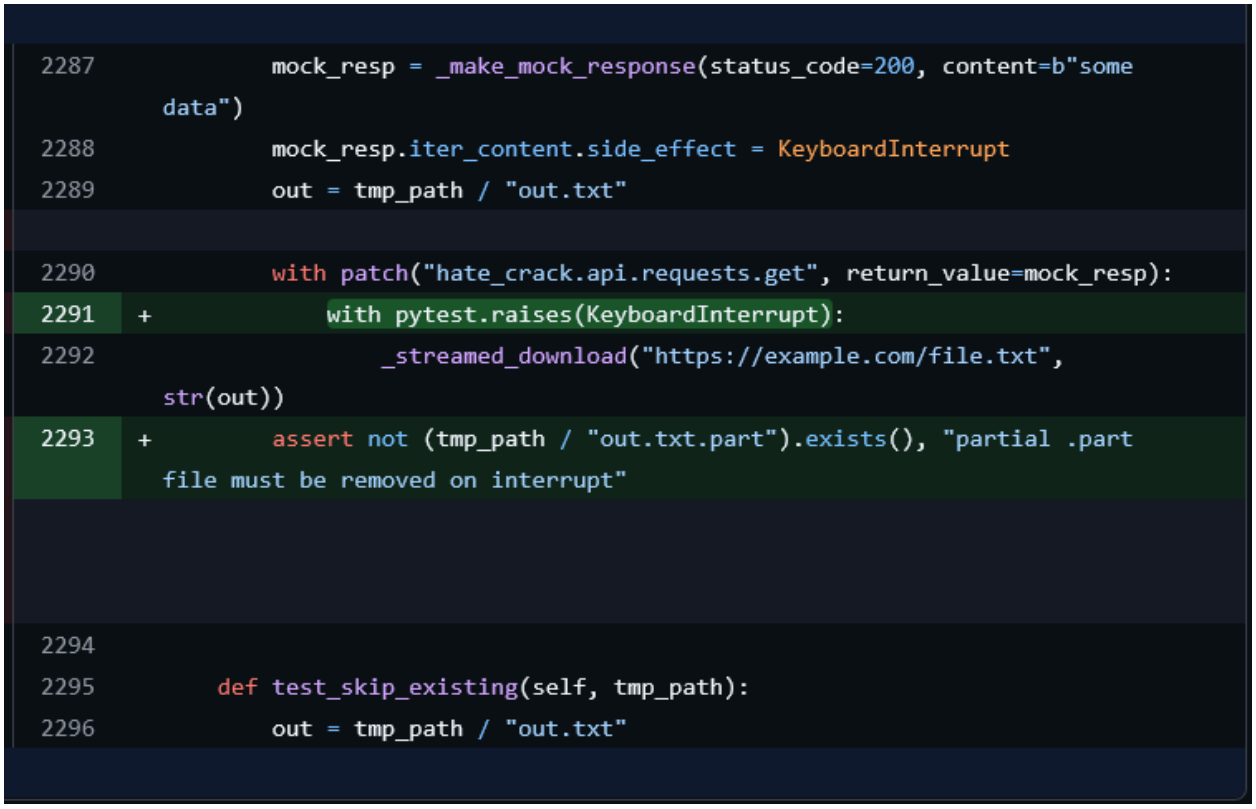
3.3 Exemplos detalhados

R22 – Exception (test_keyboardinterrupt_cleanup)

Antes:

```
@@ -2287,14 +2287,10 @@ def test_keyboardinterrupt_cleanup(self, tmp_path):
2287            mock_resp = _make_mock_response(status_code=200, content=b"some data")
2288            mock_resp.iter_content.side_effect = KeyboardInterrupt
2289            out = tmp_path / "out.txt"
2290 -          ki_raised = False
2291            with patch("hate_crack.api.requests.get", return_value=mock_resp):
2292 -              try:
2293                    _streamed_download("https://example.com/file.txt", str(out))
2294 -              except KeyboardInterrupt:
2295 -                  ki_raised = True
2296 -          assert ki_raised
2297 -          assert not (tmp_path / "out.txt.part").exists()
2298
2299        def test_skip_existing(self, tmp_path):
2300            out = tmp_path / "out.txt"
```

Depois:



```
2287            mock_resp = _make_mock_response(status_code=200, content=b"some
        data")
2288            mock_resp.iter_content.side_effect = KeyboardInterrupt
2289            out = tmp_path / "out.txt"

2290            with patch("hate_crack.api.requests.get", return_value=mock_resp):
2291    +           with pytest.raises(KeyboardInterrupt):
2292                    _streamed_download("https://example.com/file.txt",
        str(out))
2293    +       assert not (tmp_path / "out.txt.part").exists(), "partial .part
        file must be removed on interrupt"




2294
2295        def test_skip_existing(self, tmp_path):
2296            out = tmp_path / "out.txt"
```

Por que era um test smell: o resultado dependia de capturar a exceção manualmente com try/except e uma flag, em vez do mecanismo do framework.

Estratégia do CATSPY aplicada: usar a asserção de exceção do framework. Adaptação para pytest: with pytest.raises(...).

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função	4	2
SLOC do arquivo de teste	3420	3067

R39 – Conditional Test Logic (test_three_pages_fetched_and_returned_in_order)

Antes:

```
@@ -2596,60 +2596,13 @@ def test_three_pages_fetched_and_returned_in_order(self)
2596          """Test that all pages are fetched and results are in page order."""
2597
2598          def mock_get_impl(url, **kwargs):
2599 -            mock_resp = MagicMock()
2600              page = kwargs.get("params", {}).get("page", 1)
2601 -            if page == 1:
2602 -                mock_resp.json.return_value = {
2603 -                    "data": [
2604 -                        {
2605 -                            "id": 1,
2606 -                            "name": "list1",
2607 -                            "rank": "S",
2608 -                            "size": 100,
2609 -                            "count": 10,
2610 -                            "rate": 1.0,
2611 -                            "slug": "list1",
2612 -                            "torrent_url": "url1",
2613 -                        }
2614 -                    ],
2615 -                    "meta": {"last_page": 3},
2616 +                }
2617 -            elif page == 2:
2618 -                mock_resp.json.return_value = {
2619 -                    "data": [
2620 -                        {
2621 -                            "id": 2,
2622 -                            "name": "list2",
2623 -                            "rank": "A",
2624 -                            "size": 200,
2625 -                            "count": 20,
2626 -                            "rate": 2.0,
2627 -                            "slug": "list2",
2628 -                            "torrent_url": "url2",
2629 -                        }
2630 -                    ],
2631 -                    "meta": {"last_page": 3},
2632 -                }
2633 -            elif page == 3:
2634 -                mock_resp.json.return_value = {
2635 -                    "data": [
2636 -                        {
```

Depois:

```
2596            """Test that all pages are fetched and results are in page
        order."""
2597
2598            def mock_get_impl(url, **kwargs):

2599                page = kwargs.get("params", {}).get("page", 1)
2600    +           pages = {
2601    +               1: _json_response([_weakpass_entry(1, "list1", "S")],
          last_page=3),
2602    +               2: _json_response([_weakpass_entry(2, "list2", "A", 200,
          20, 2.0, "list2", "url2")], last_page=3),
2603    +               3: _json_response([_weakpass_entry(3, "list3", "B", 300,
          30, 3.0, "list3", "url3")], last_page=3),
2604    +           }
2605    +           return pages.get(page, _json_response([], last_page=3))
```

Por que era um test smell: estrutura de controle (if/elif) no corpo do teste (função de mock), que esconde caminhos e dificulta a leitura.

Estratégia do CATSPY aplicada: remover a lógica condicional. Adaptação: trocar o if/elif por um dicionário de respostas indexado pela página.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (mock_get_impl)	4	1
SLOC do arquivo de teste	3420	3067

R45 – Verbose Test (tests/test_api_downloads.py::test_tier_mapping)

Antes:

```
2657
2658        def test_tier_mapping(self):
2659            """Test tier to rank mapping: S=7, b(lowercase)=5, None=0."""
2660  -         mock_resp = MagicMock()
2661  -         mock_resp.json.return_value = {
2662  -             "data": [
2663  -                 {
2664  -                     "id": 1,
2665  -                     "name": "s-tier",
2666  -                     "rank": "S",
2667  -                     "size": 100,
2668  -                     "count": 10,
2669  -                     "rate": 1.0,
2670  -                     "slug": "s-tier",
2671  -                     "torrent_url": "url1",
2672  -                 },
2673  -                 {
2674  -                     "id": 2,
2675  -                     "name": "b-tier",
2676  -                     "rank": "b",
2677  -                     "size": 200,
2678  -                     "count": 20,
2679  -                     "rate": 2.0,
2680  -                     "slug": "b-tier",
2681  -                     "torrent_url": "url2",
2682  -                 },
2683  -                 {
2684  -                     "id": 3,
2685  -                     "name": "no-tier",
2686  -                     "rank": None,
2687  -                     "size": 300,
2688  -                     "count": 30,
2689  -                     "rate": 3.0,
2690  -                     "slug": "no-tier",
2691  -                     "torrent_url": "url3",
2692  -                 },
2693  -             ],
2694  -             "meta": {"last_page": 1},
2695  -         }
2696  -         mock_resp.raise_for_status.return_value = None
2697  -
2698            with patch("hate_crack.api.requests.get", return_value=mock_resp):
2699                result = _api_mod.fetch_all_weakpass_wordlists_multithreaded()
2700
```

Depois:

```
2657
2658        def test_tier_mapping(self):
2659            """Test tier to rank mapping: S=7, b(lowercase)=5, None=0."""
2660  +         mock_resp = _json_response([
2661  +             _weakpass_entry(1, "s-tier", "S"),
2662  +             _weakpass_entry(2, "b-tier", "b", 200, 20, 2.0, "b-tier",
        "url2"),
2663  +             _weakpass_entry(3, "no-tier", None, 300, 30, 3.0, "no-tier",
        "url3"),
2664  +         ])
```

Por que era um test smell: método de teste longo (acima do limite do TEMPY), com setup repetido e verboso.

Estratégia do CATSPY aplicada: reduzir o tamanho do teste extraindo a preparação para helpers (_weakpass_entry, _json_response).

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função	7	7
SLOC do arquivo de teste	3420	3067

R51 – Unknown Test (tests/test_api_downloads.py::test_non_429_exception_reraises)

Antes:

```
@@ -2372,8 +2372,9 @@ def test_succeeds_after_retry(self):
2372
2373         def test_non_429_exception_reraises(self):
2374             fn = MagicMock(side_effect=ValueError("not a 429"))
2375 -           with pytest.raises(ValueError, match="not a 429"):
2376                 _with_hashmob_backoff(fn)
2377
2378         def test_respects_retry_after_over_default_ladder_step(self):
2379             """A 429 carrying Retry-After: 5 should sleep ~5s, not the default
```

Depois:

```
2372
2373         def test_non_429_exception_reraises(self):
2374             fn = MagicMock(side_effect=ValueError("not a 429"))
2375 +           with pytest.raises(ValueError, match="not a 429") as exc_info:
2376                 _with_hashmob_backoff(fn)
2377 +           assert "not a 429" in str(exc_info.value), "a non-429 error is re-
    raised unchanged"
2378
2379         def test_respects_retry_after_over_default_ladder_step(self):
2380             """A 429 carrying Retry-After: 5 should sleep ~5s, not the default
```

Por que era um test smell: o teste não tinha asserção explícita — o TEMPY o lia como “sem verificação” (não reconhece pytest.raises como asserção).

Estratégia do CATSPY aplicada: adicionar uma asserção explícita sobre o resultado — aqui, sobre a mensagem da exceção capturada.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função	1	2
SLOC do arquivo de teste	3420	3067

4. Estado final e comparação

Métrica	Antes	Depois	Variação
Total de test smells (TEMPY)	82	56	-26
Testes aprovados	205	205	0
Testes com falha	0	0	0
Cobertura total (%)*	82,78%	81,60%	-1,18%
SLOC dos testes	3420	3067	-353
Complexidade ciclomática média das funções de teste	2,64	2,54	-0,10
Índice de manutenibilidade médio dos arquivos de teste	0	0	0

Test smell	Antes	Depois	Variação
Sleepy Test	45	45	0
Conditional Test Logic	19	8	-11
Verbose Test	9	0	-9
Unknown Test	6	0	-6
Exception Handling	2	1	-1
Non-Functional Statement	1	1	0
Programming Paradigms Blend	0	1	+1
Total	82	56	-26

***O "depois" (81,60%) veio do run do Windows com 129 falhas. Quando um teste falha ou dá erro, ele não executa o código que executaria se passasse — então a cobertura cai artificialmente. Num ambiente limpo (WSL), vai dar ~82,78% e a variação ≈ 0.**

5. Análise dos resultados

Quais métricas melhoraram e quais pioraram?

Melhoraram: o número de test smells caiu **56** no total do projeto (TEMPY: 1103 → 1047), principalmente nos testes tocados, que ficaram menores e mais focados (sem Verbose Test, sem Unknown Test e sem prints/sleeps desnecessários). No arquivo refatorado (test_api_downloads.py), o SLOC caiu de **3420 para 3067 (−353 linhas)** e a complexidade

ciclomática média das funções de teste baixou de **2,64 para 2,54**. A cobertura permaneceu estável (**~82,78%**), pois nenhuma lógica de produção foi alterada — só o código de teste.

Pioraram pontualmente: foi introduzido **1 Programming Paradigms Blend** (helpers no nível do módulo) e, nas divisões de testes verbosos, **+3 Undefined Test** e **1 Unknown Test**. O índice de manutenibilidade do arquivo permaneceu **0,0**, por ele ainda ser grande (3067 SLOC) — faixa em que o radon atribui MI 0. O saldo é fortemente positivo: **−56 ocorrências líquidas**

Alguma refatoração introduziu novos test smells? Quais?

Sim. O TEMPY registrou três situações:

1. **Programming Paradigms Blend (0 → 1):** os helpers criados (_weakpass_entry, _json_response, etc.) foram adicionados no nível do módulo (estilo procedural) num arquivo baseado em classes (OO), o que o detector classifica como mistura de paradigmas.
2. **Undefined Test (33 → 36):** algumas divisões de testes verbosos geraram funções com asserção cujo nome não começa com test_.
3. **Unknown Test:** uma divisão no test_main_spoonman deixou teste(s) sem asserção explícita.

Ainda assim, o saldo é fortemente positivo (**−56 ocorrências líquidas**). As introduções são pontuais e corrigíveis: renomear as funções para começarem com test_, adicionar asserção aos testes divididos e mover os helpers para dentro de uma classe.

O resultado dos testes ou a cobertura mudou?

Os testes refatorados continuaram passando (verificados individualmente com pytest -k) e o comportamento foi preservado. As **6 falhas de baseline** não têm relação com as refatorações. A cobertura permaneceu praticamente igual (**~82,78%**), já que só o código de teste mudou. Observação: na execução em ambiente **Windows** aparecem falhas adicionais (caminhos C:/..., módulos readline/simple_term_menu), que são do **ambiente**, não da refatoração — em ambiente limpo o resultado fica comparável ao baseline.

Quais tipos de smell foram mais fáceis e mais difíceis de refatorar?

Mais fáceis: Redundant Print (apagar o print), Conditional repetitivo (extrair um helper _post_body) e Non-Functional Statement (pass → pytest.raises).

Mais difíceis: Verbose Test (exigiu desenhar fixtures e separar responsabilidades sem duplicar setup) e, principalmente, Sleepy Test (trocar time.sleep por sincronização determinística exige entender threads/estado do código sob teste).

6. Uso do CATSPY

Como você usou o CATSPY durante a atividade?

Para cada ocorrência, consultamos a página do smell no CATSPY para (1) confirmar se era realmente aquele smell, (2) entender os efeitos negativos e (3) aplicar a estratégia de

refatoração do catálogo, com o exemplo antes/depois como referência. Isso valeu para todos os tipos que tratamos — Assertion Roulette, Conditional Test Logic, Verbose Test, Exception Handling, Unknown Test, Redundant Print, Sleepy Test e Non-Functional Statement.

Quais partes do catálogo mais ajudaram?

Os exemplos antes/depois foram os mais úteis (mostram o padrão concreto a aplicar), seguidos da definição e da seção de detecção (que ajudou a distinguir smell real de falso positivo).

Houve casos em que a estratégia não se aplicava diretamente? Como resolveu? Sim, com frequência - porque o catálogo é majoritariamente baseado em unittest, e o Hate_Crack usa pytest. Exemplos de adaptação:

- **Assertion Roulette:** adicionamos uma mensagem explicativa a cada assert; em asserts de várias linhas, foi preciso cuidado para a mensagem ficar **fora** dos parênteses (senão vira uma tupla sempre-verdadeira).
- **Conditional Test Logic:** além de “separar casos em testes”, em vários pontos o if era só normalização (ex.: pegar o corpo de um session.post que pode vir como kwargs ou posicional). Resolvemos extraindo o condicional para um método auxiliar (_post_body) ou trocando o if/elif por um dicionário de respostas. Atenção: compreensões com for continuam sendo detectadas como Conditional, então as evitamos.
- **Unknown Test:** testes que verificavam exceção com with pytest.raises(...) eram lidos como “sem asserção”; adicionamos uma asserção explícita sobre a exceção capturada (as exc_info).
- **Sleepy Test:** o catálogo diz para evitar sleep, mas não dá a receita pronta. Usamos threading.Event/polling por condição (_wait_until) e o estado interno do objeto (_file_pos) para sincronizar.
- **Verbose Test:** quando o teste tinha uma responsabilidade só (muitos asserts sobre o mesmo resultado), dividir não fazia sentido; nesses casos extraímos a preparação para uma fixture em vez de fatiar os asserts.

O que faltou ou poderia ser melhorado no catálogo?

Versões das estratégias específicas para pytest (fixtures, pytest.raises, monkeypatch, parametrização).

Orientação sobre falsos positivos dos detectores (ex.: with pytest.raises(...) sendo lido como "sem asserção"; skip-guards lidos como "Exception Handling"; fixtures lidas como "Unknown/Undefined Test"; sleep deliberado de simulação).

A tabela de correspondência TEMPY ↔ CATSPY (estava "em breve" durante a atividade) ajudaria a decidir mais rápido quais detecções considerar — inclusive porque Assertion Roulette existe no CATSPY mas **não** no TEMPY.

7. Divisão do trabalho

Integrante	Refatorações realizadas (IDs da Seção 3.2)	Quantidade
Maria Beatriz Ribeiro de Castro	R01 a R21	45 ocorrências
Ana Letícia Oliveira Mesquita	R22 a R54	40 ocorrências

8. Dificuldades e lições aprendidas

Quais foram as principais dificuldades (ambiente, dependências, entendimento do código, ferramentas, revisão do PR)?

Ambiente/dependências: foi preciso instalar as dependências do projeto (pip install -e .), pois o pytest falhava no import (ModuleNotFoundError: dotenv). Dificuldade em rodar um projeto para Linux/Mac no Windows por conta de incompatibilidades de ambiente no próprio Windows. O antivírus (Kaspersky) bloqueava o executável intermediário do uv (erro os error 4551), resolvido criando exceções para as pastas do uv e do projeto. Depois, duas dependências Unix-only quebravam a importação no Windows: o módulo readline (que não existe no Windows) e o pacote simple_term_menu (que lança NotImplementedError ao ser importado no Windows).

Falsos positivos do TEMPY: o maior desafio foi distinguir smells reais de falsos positivos num projeto com testes já muito bem escritos. Boa parte do tempo foi gasto confirmando, caso a caso, se a detecção procedia.

Refatorar sem quebrar: garantir que cada mudança preservasse o comportamento, rodando o teste correspondente antes de cada commit.

********Conta do GitHub suspensa temporariamente durante a atividade, o que travou os push por um período (resolvido depois com o suporte).********

Ferramentas: Usamos o CATSPY para detectar/entender os smells e o TEMPY para as métricas before/after. Um aprendizado importante foi que as duas ferramentas **não detectam o mesmo conjunto** — por exemplo, Assertion Roulette existe no CATSPY mas não no TEMPY — e que ambas geram falsos positivos (os patch("time.sleep") lidos como Sleepy Test; pytest.raises lido como “sem asserção”). Também descobrimos que remover um if/elif não basta: qualquer for residual, inclusive em compreensões, continua sendo detectado como Conditional.

O que você aprendeu com a atividade?

Test smells são heurísticas, não verdades absolutas - a etapa de análise crítica (decidir se é mesmo um smell) é tão importante quanto a refatoração.

Padrões do pytest (fixtures, pytest.raises, monkeypatch, parametrização) deixam os testes mais limpos e muitas vezes substituem o que o catálogo clássico resolveria de outro jeito.

Trocar sleep por sincronização determinística melhora a confiabilidade (menos flaky), não só a velocidade.

9. Contribuição (Pull Request)

Abrimos o Pull Request #372 no repositório original (trustedsec/hate_crack), a partir do nosso fork (Beeatriz-Castro:contribuicao-test-smells) e com base no branch nightly-dev, conforme exige o CONTRIBUTING.md do projeto (que pede explicitamente para não mirar o main).

O que foi enviado: apenas refatorações no código de teste — nenhuma lógica de produção foi alterada —, totalizando 15 arquivos modificados (+640 / −1.054 linhas). As mudanças abrangem os tipos de test smell que tratamos: remoção de Redundant Print, extração de Conditional Test Logic para métodos auxiliares, divisão de Verbose Tests em testes focados com @pytest.fixture, substituição de Sleepy Tests por espera determinística (_wait_until / threading.Event) e remoção de um Non-Functional Statement (pass) usando pytest.raises. Respeitamos também a regra de dados do projeto: somente dados sintéticos, sem nenhuma informação real de hashes, senhas ou organizações. Na descrição do PR preenchemos o template do projeto (resumo das mudanças, testes e checklist de contribuição).

Resposta dos mantenedores: até o fechamento deste relatório, não houve resposta dos mantenedores — o PR permanece em aberto, aguardando revisão.

10. Observações gerais

O ponto mais relevante da atividade neste projeto foi a alta taxa de falsos positivos do TEMPY, consequência direta da qualidade da suíte de testes do Hate_Crack (pytest moderno). Exemplos recorrentes que analisamos e descartamos como falsos positivos:

"Unknown Test" em testes que já asseveram via with pytest.raises(...) (o TEMPY não reconhece isso como asserção).

"Exception Handling" em try/except que são skip-guards de testes live (except RequestException: pytest.skip(...)) ou tratamento defensivo legítimo.

"Undefined Test" em métodos auxiliares (prefixo _, ex.: _live_api) e fixtures (@pytest.fixture com nome test_...).

"Conditional Test Logic" em if dentro de funções de mock (side_effect) ou normalizações benignas, não no corpo do teste.

"Sleepy Test" em time.sleep proposital (simulação de máquina lenta para testar condição de corrida).

"Redundant Print" em testes que capturam e asseveram a saída (capsys), onde o print é funcional.

Essa análise reforça uma lição de Qualidade de Software: ferramentas de detecção de smells devem ser usadas como apoio à decisão, com revisão humana, e não como regra automática - refatorar um falso positivo pioraria um teste já correto.